UIT2702 - Cloud and Distributed Computing
Exercise 1: Inter-Process Communication in Distributed Computing

AIM:

To implement and understand inter-process communication using sockets in a distributed client-server setup.

PROCEDURE:

Task 1 - Setup Basic Client-Server Communication:

- Create a simple socket connection where the client sends a message and server responds.
- On server side, bind to a host and port, listen for client connections, and accept messages.
- Implement message handling where the server parses and sums numbers.

Task 2 - Enhanced Communication with Object Serialization:

- Define a custom data object (DataObject) with attributes (name and values) and enable object-based data transfer.
- Serialize and deserialize objects using Python's pickle module.
- Send a DataObject from client to server, and compute the sum of values on the server.
- Send a response from the server back to the client, indicating the sum of values.

CODE:

client1.py

```
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((socket.gethostname(), 12345))
while True:
    message = input("Send message to server: ")
    try:
        if message.lower().strip() != 'exit':
            client_socket.send(message.encode())
            data = client_socket.recv(1024).decode()
            print(f"Message received from server: {data}")
        else:
            break
    except ConnectionAbortedError as e:
        print(e)
client_socket.close()
```

server1.py

```
import socket
```

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((socket.gethostname(), 12345))
server_socket.listen()
client_socket, addr = server_socket.accept()
print(f"Connected {client_socket} on {addr}")
while True:
    message = client_socket.recv(1024).decode()
    try:
        if not message:
            print("No message received. Client may have closed the connection.")
            break
        if message.lower().strip() == 'exit':
            print("Received 'exit' message. Closing connection.")
            break
        print(f"Message received from client: {message}")
        list1 = message.split()
        total_sum = 0
        for i in list1:
            total_sum += int(i)
        response = client_socket.send(str(total_sum).encode())
        print(f"Sending data {total_sum} to client")
    except ConnectionAbortedError as e:
        print(e)
client_socket.close()
server_socket.close()
```

client2.py

```
import socket
import pickle
class DataObject:
    def __init__(self, name, values):
        self.name = name
        self.values = values
    def __str__(self):
        return f"DataObject(name={self.name}, values={self.values})"
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((socket.gethostname(), 12345))
name = input("Enter name: ")
values = eval(input("Enter list of values: "))
data_to_send = DataObject(name=name, values=values)
print(f"Sending object: {data_to_send}")
serialized_data = pickle.dumps(data_to_send)
client_socket.send(serialized_data)
response = client_socket.recv(1024).decode()
print(f"Received response from server: {response}")
client_socket.close()
```

server2.py

```python
import socket
import pickle
class DataObject:
    def __init__(self, name, values):
        self.name = name
        self.values = values
    def __str__(self):
        return f"DataObject(name={self.name}, values={self.values})"
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((socket.gethostname(), 12345))
server_socket.listen()
client_socket, addr = server_socket.accept()
print(f"Connected to {addr}")
while True:
    try:
        received_data = client_socket.recv(1024)
        if not received_data:
            print("No data received. Closing connection.")
            break
        data_object = pickle.loads(received_data)
        print(f"Received object from client: {data_object}")
        total_sum = sum(data_object.values)
        response = f"Server: Received {data_object.name}'s data. Sum = {total_sum}"
        client_socket.send(response.encode())
    except Exception as e:
        print(f"Error: {e}")
        break
client_socket.close()
server_socket.close()
```

SAMPLE INPUT AND OUTPUT:

```
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex1> python3 se
rver1.py
Connected <socket.socket fd=1072, family=2, type=1, proto=0, lad
dr=('10.121.32.119', 12345), raddr=('10.121.32.119', 50932)> on
('10.121.32.119', 50932)
Message received from client: 1 2 3 4 5
Sending data 15 to client
Message received from client: 12 1 10
Sending data 23 to client
No message received. Client may have closed the connection.
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex1>
```

```
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex1> python3 cl
ient1.py
Send message to server: 1 2 3 4 5
Message received from server: 15
Send message to server: 12 1 10
Message received from server: 23
Send message to server: exit
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex1>
```

```
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex1> python3 se
rver2.py
Connected to ('10.121.32.119', 51852)
Received object from client: DataObject(name=selcia, values=[1,
2, 3])
No data received. Closing connection.
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex1>
```

```
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex1> python3 cl
ient2.py
Enter name: selcia
Enter list of values: [1,2,3]
Sending object: DataObject(name=selcia, values=[1, 2, 3])
Received response from server: Server: Received selcia's data. S
um = 6
```

RESULT:

Successfully established a basic client-server communication framework, later enhanced to support serialized data transfer and custom data handling.

Exercise 2: Remote Method Invocation (RMI)

AIM:

To implement Remote Method Invocation (RMI) to facilitate remote execution of methods in a distributed computing setup.

PROCEDURE:

1. **Define Remote Service**:
    - Create a Calculator class on the server with methods for addition and multiplication.
    - Use Pyro4 to expose these methods for remote invocation by clients.
2. **Server Setup**:
    - Register the Calculator service with a Pyro4.Daemon to generate a unique URI.
    - Start the daemon to listen for incoming client requests.
3. **Client Invocation**:
    - Prompt the client to input the server URI, connect via Pyro4.Proxy, and invoke the remote add_numbers and multiply methods.
    - Display the results of both operations on the client side.

CODE:

client.py

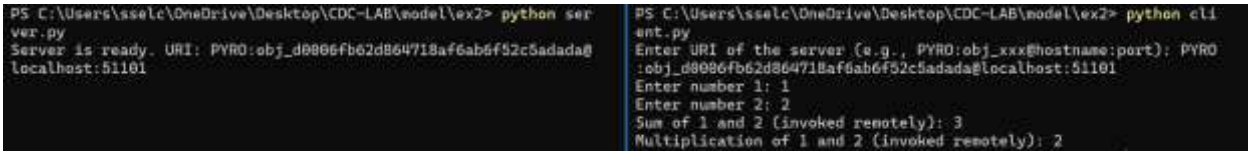
```
import Pyro4
def call_remote_service():
    uri = input("Enter URI of the server (e.g., PYRO:obj_xxx@hostname:port): ")
    remote_calculator = Pyro4.Proxy(uri)
    a = int(input("Enter number 1: "))
    b = int(input("Enter number 2: "))
    sum_result = remote_calculator.add_numbers(a, b)
    mul_result = remote_calculator.multiply(a, b)
    print(f"Sum of {a} and {b} (invoked remotely): {sum_result}")
    print(f"Multiplication of {a} and {b} (invoked remotely): {mul_result}")
if __name__ == "__main__":
    call_remote_service()
```

server.py

```
import Pyro4
@Pyro4.expose
class Calculator:
```

```
    def add_numbers(self, a, b):
        return a+b
    def multiply(self, a, b):
        return a*b
def start_server():
    daemon = Pyro4.Daemon()
    uri = daemon.register(Calculator)
    print(f"Server is ready. URI: {uri}")
    daemon.requestLoop()
if __name__ == "__main__":
    start_server()
```

SAMPLE INPUT AND OUTPUT:



RESULT:

Successfully created a distributed application where clients invoke remote methods on a server to perform calculations.

UIT2702 - Cloud and Distributed Computing
Exercise 3: Message Passing and Distributed Mutual Exclusion

AIM:

To implement message passing and distributed mutual exclusion using sockets and the Ricart-Agrawala algorithm.

PROCEDURE:

Task 1 - Message Passing System:

- Create a server that listens for connections and allows multiple clients to connect via sockets.
- Each client can send and receive messages, enabling communication between distributed processes.

Task 2 - Distributed Mutual Exclusion (Ricart-Agrawala Algorithm):

- Implement the Ricart-Agrawala algorithm to manage mutual exclusion across multiple processes.
- Each process sends requests with timestamps and waits for replies to enter the critical section.
- After performing the task, each process releases the critical section and resets.

CODE:

client.py

```
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((socket.gethostname(), 12345))
print("Connected to the server. Type 'exit' to disconnect.")
while True:
    try:
        message = input("Enter message: ")
        client_socket.send(message.encode())
        if message.lower().strip() == 'exit':
            print('Closing connection')
            break
        response = client_socket.recv(1024).decode()
        print(f"Message received: {response}")
    except Exception as e:
        print(f"Error: {e}")
client_socket.close()
```

server.py

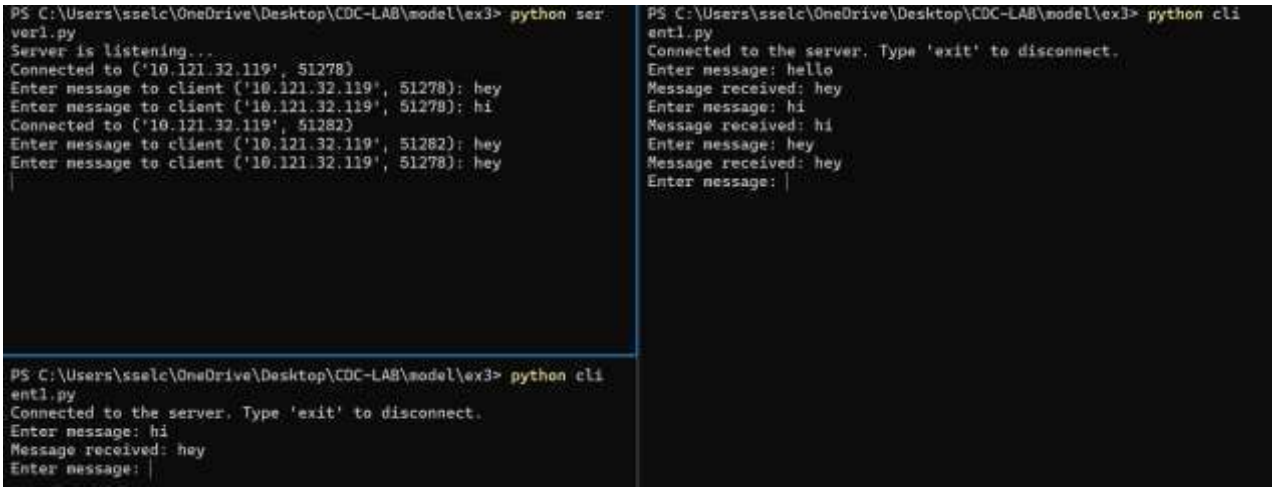
```
import socket
import threading
def handle_client(client_socket, addr):
    print(f"Connected to {addr}")
    while True:
        try:
            message = client_socket.recv(1024).decode()
            if not message or message.lower().strip() == 'exit':
                print(f"Client {addr} disconnected...")
                break
            response = input(f'Enter message to client {addr}: ')
            client_socket.send(response.encode())
        except Exception as e:
            print(f"Error: {e}")
    print(f"Connection closed with {addr}")
    client_socket.close()
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((socket.gethostname(), 12345))
server_socket.listen()
print("Server is listening...")
while True:
    client_socket, addr = server_socket.accept()
    client_handler = threading.Thread(target=handle_client, args=(client_socket, addr))
    client_handler.start()
```

ricart-agrawala.py

```
import threading
import time
import random
class Process:
    def __init__(self, id, total_processes):
        self.id = id
        self.total_processes = total_processes
        self.timestamp = 0
        self.requested = False
        self.reply_count = 0
        self.lock = threading.Lock()
        self.request_limit = 3  # Limit the number of requests per process
        self.request_count = 0  # Count how many requests have been made
    def request_critical_section(self):
        if self.request_count < self.request_limit:
            self.timestamp += 1
            self.requested = True
            print(f"Process {self.id} requesting critical section at timestamp {self.timestamp}.")
            for i in range(self.total_processes):
                if i != self.id:
```

```
                    processes[i].receive_request(self.id, self.timestamp)
            while self.reply_count < self.total_processes - 1:
                time.sleep(0.1)
            self.enter_critical_section()
            self.request_count += 1  # Increment the request count
        else:
            print(f"Process {self.id} has reached its request limit and will terminate.")
    def receive_request(self, sender_id, sender_timestamp):
        with self.lock:
            if self.requested and self.timestamp < sender_timestamp:
                    print(f"Process {self.id} queuing request from Process {sender_id} at timestamp
{sender_timestamp}.")
                return
            else:
                print(f"Process {self.id} sending reply to Process {sender_id}.")
                processes[sender_id].receive_reply(self.id)
    def receive_reply(self, sender_id):
        with self.lock:
            self.reply_count += 1
            print(f"Process {self.id} received reply from Process {sender_id}.")
    def enter_critical_section(self):
        print(f"Process {self.id} entering critical section.")
        time.sleep(random.uniform(1, 3))  # Simulate work in critical section
        print(f"Process {self.id} exiting critical section.")
        self.release_critical_section()
    def release_critical_section(self):
        self.requested = False
        self.reply_count = 0
        print(f"Process {self.id} released critical section.")
def process_runner(process):
    while process.request_count < process.request_limit:
        time.sleep(random.uniform(1, 2))
        process.request_critical_section()
num_processes = 5
processes = [Process(i, num_processes) for i in range(num_processes)]
threads = []
for process in processes:
    t = threading.Thread(target=process_runner, args=(process,))
    threads.append(t)
    t.start()
for t in threads:
    t.join()
print("All processes have completed their requests.")
```

SAMPLE INPUT AND OUTPUT:



RESULT:

Successfully implemented message passing and distributed mutual exclusion, allowing synchronized resource access among distributed processes.

UIT2702 - Cloud and Distributed Computing
Exercise 4: Message Passing and Distributed Mutual Exclusion

AIM:

To implement logical clocks and vector clocks to manage event ordering and causality in distributed systems.

PROCEDURE:

Task 1 - Logical Clocks (Lamport's Logical Clock):

- Initialize each process with a logical clock.
- For each event or message sent, increment the clock and send the timestamp.
- On message receipt, update the clock based on the received timestamp, ensuring ordered events.

Task 2 - Vector Clocks:

- Set up each process with a vector representing the state of all processes.
- For each event or message sent, update the process's vector and transmit it.
- Upon receipt, merge the vectors to reflect concurrent events and causality relationships.

CODE:

lamport_clock.py

```
class LamportClock:
    def __init__(self, process_id):
        self.process_id = process_id
        self.clock = 0
    def event(self):
        self.clock += 1
        print(f"Process {self.process_id} - Internal event with Timestamp {self.clock}")
    def send_message(self, receiver):
        self.clock += 1
          print(f"Process {self.process_id} sends a message to Process {receiver.process_id} with
Timestamp {self.clock}")
        receiver.receive_message(self.process_id, self.clock)
    def receive_message(self, sender_id, sender_timestamp):
        self.clock = max(self.clock, sender_timestamp) + 1
         print(f"Process {self.process_id} received a message from Process {sender_id} with updated
Timestamp {self.clock}")
process1 = LamportClock(1)
process2 = LamportClock(2)
process1.event()
```

```
process1.send_message(process2)
process2.event()
process2.send_message(process1)
```

vector_clock.py

```
class VectorClock:
    def __init__(self, process_id, num_processes):
        self.process_id = process_id
        self.num_processes = num_processes
        self.vector = [0] * num_processes
    def event(self):
        self.vector[self.process_id] += 1
        print(f"Process {self.process_id} - Internal event with Updated Vector {self.vector}")
    def send_message(self, receiver):
        self.vector[self.process_id] += 1
          print(f"Process {self.process_id} sends a message to Process {receiver.process_id} with
Updated Vector {self.vector}")
        receiver.receive_message(self.process_id, self.vector)
    def receive_message(self, sender_id, sender_vector):
        for i in range(self.num_processes):
            self.vector[i] = max(self.vector[i], sender_vector[i]) + 1
        self.vector[self.process_id] += 1
         print(f"Process {self.process_id} received a message from Process {sender_id} with Updated
Vector {self.vector}")
# Create Processes
process1 = VectorClock(0, 3)
process2 = VectorClock(1, 3)
process3 = VectorClock(2, 3)
# Simulate Events
process1.event()
process1.send_message(process2)
process2.event()
process2.send_message(process3)
process3.send_message(process1)
```

SAMPLE INPUT AND OUTPUT:

```
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex4> python .\lamport_clock.py
Process 1 - Internal event with Timestamp 1
Process 1 sends a message to Process 2 with Timestamp 2
Process 2 received a message from Process 1 with updated Timestamp 3
Process 2 - Internal event with Timestamp 4
Process 2 sends a message to Process 1 with Timestamp 5
Process 1 received a message from Process 2 with updated Timestamp 6
```

```
PS C:\Users\sselc\OneDrive\Desktop\CDC-LAB\model\ex4> python .\vector_clock.py
Process 0 - Internal event with Updated Vector [1, 0, 0]
Process 0 sends a message to Process 1 with Updated Vector [2, 0, 0]
Process 1 received a message from Process 0 with Updated Vector [3, 2, 1]
Process 1 - Internal event with Updated Vector [3, 3, 1]
Process 1 sends a message to Process 2 with Updated Vector [3, 4, 1]
Process 2 received a message from Process 1 with Updated Vector [4, 5, 3]
Process 2 sends a message to Process 0 with Updated Vector [4, 5, 4]
Process 0 received a message from Process 2 with Updated Vector [6, 6, 5]
```

RESULT:

Successfully implemented logical and vector clocks, allowing consistent event ordering and causality detection across distributed processes.

UIT2702 - Cloud and Distributed Computing
Exercise 5: Virtual Instances and Server Consolidation with Fault Tolerance in a Cloud Environment

AIM:

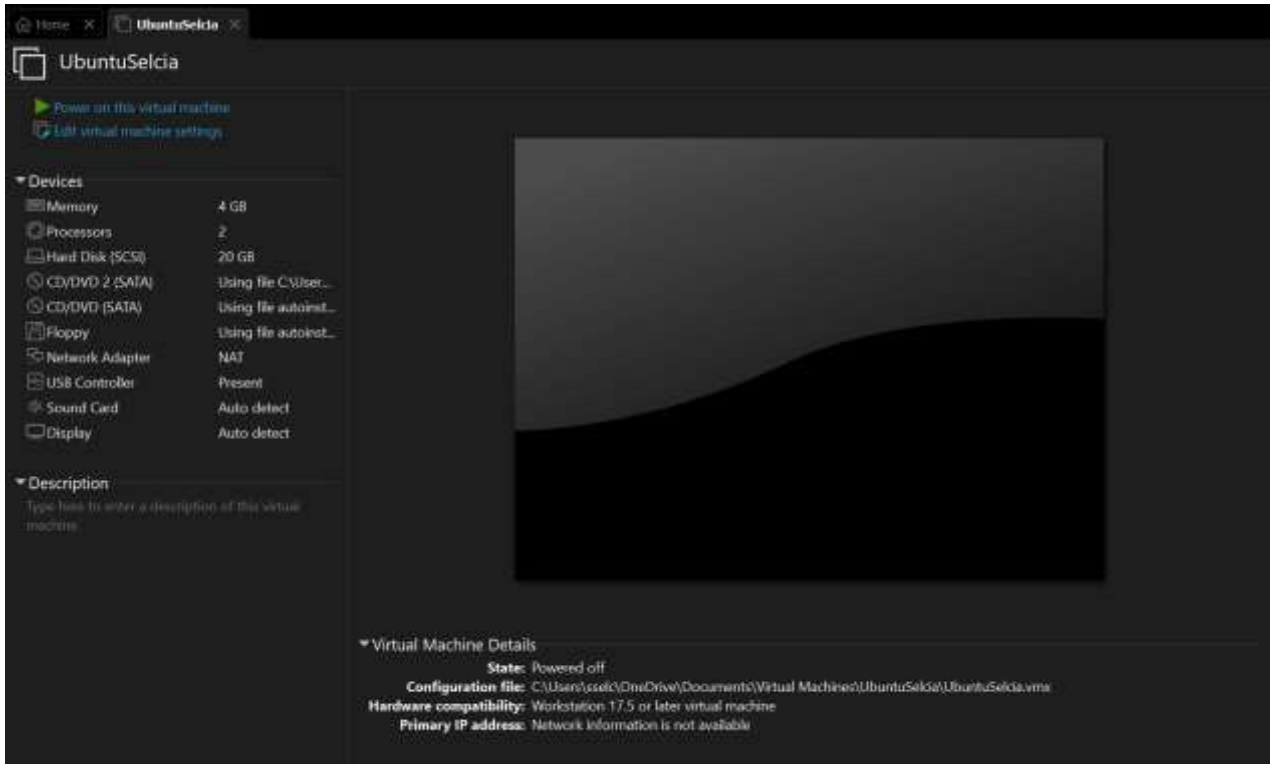
To set up virtual instances, load balancing, server consolidation, and fault tolerance to optimize resource use and ensure high availability in a cloud environment.

PROCEDURE:

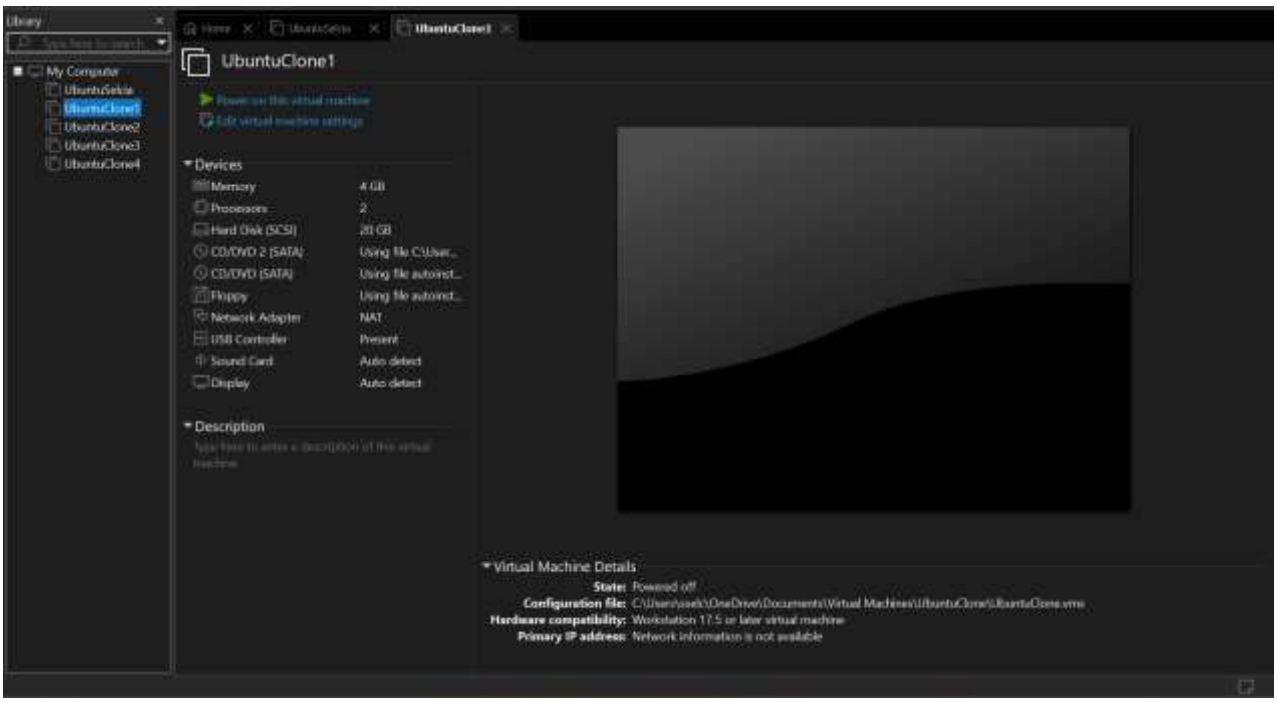
Part 1: Virtual Instances Management

Task 1: Create and Configure Virtual Instances

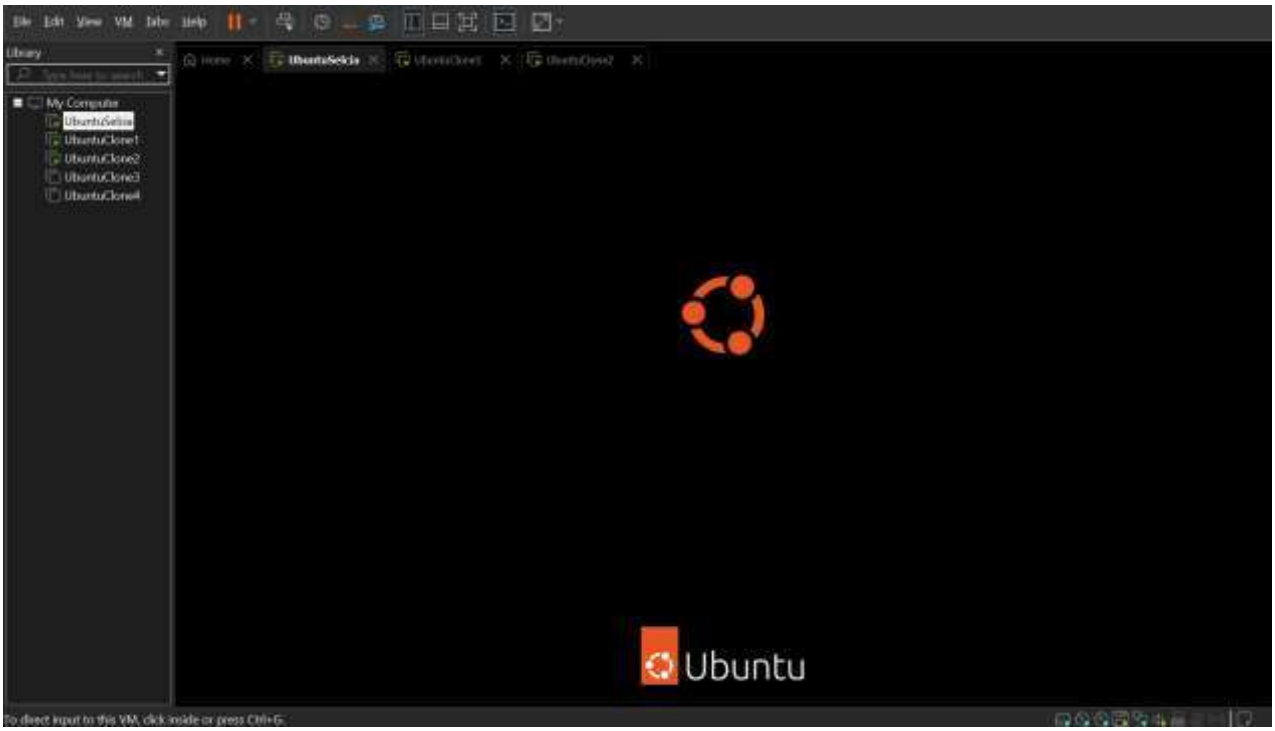
1. **Open VMware Workstation Pro 17:**
    - Start VMware Workstation and open the existing Ubuntu 24.04.1 LTS VM.



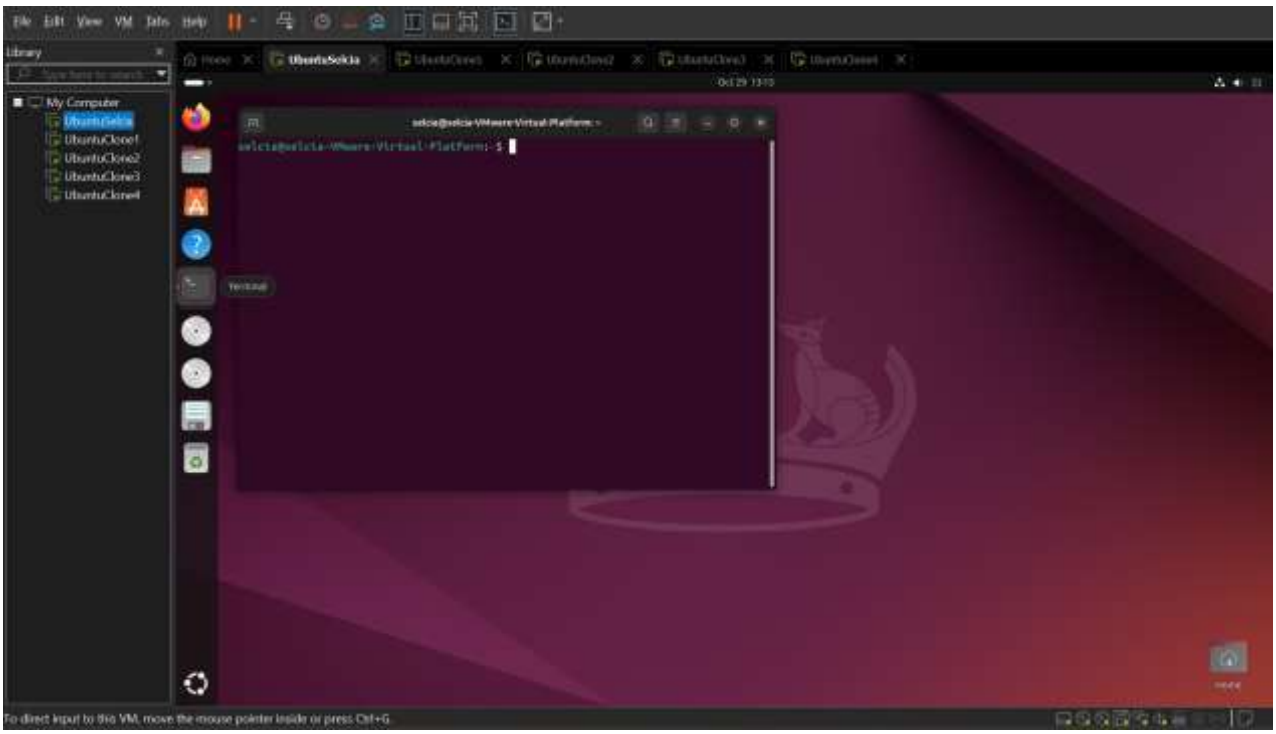
2. **Clone Virtual Instances (VMs):**
    - Since we already have one Ubuntu VM, we can create additional VMs by cloning.
    - **Right-click** on the existing Ubuntu VM and select **Manage** > **Clone**.
    - In the cloning wizard, select **Create a full clone** to make identical copies of VM.
    - Clone as many VMs as needed (e.g., two or three additional VMs).



3. **Power On the Cloned VMs:**
    - Power on the newly cloned VMs by right-clicking on each & select **Power On**.



    - Once powered on, open the terminal in each VM.



4. **Install Apache Web Server on Each VM:**

For each VM, install Apache using the following commands: sudo apt update

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo apt update
[sudo] password for selcia:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
31 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

sudo apt install apache2 -y

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libaprit64 libaprutil1-dbd-sqlite3
  libaprutil1-ldap libaprutil1t64
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libaprit64
  libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64
0 upgraded, 8 newly installed, 0 to remove and 31 not upgraded.
Need to get 1,900 kB of archives.
After this operation, 7,455 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 libaprit64 amd64 1.7.2-3.1ubuntu0.1 [108 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1t64 amd64 1.6.3-1.1ubuntu7 [91.9 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1-dbd-sqlite3 amd64 1.6.3-1.1ubuntu7 [11.2 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu noble/main amd64 libaprutil1-ldap amd64 1.6.3-1.1ubuntu7 [9,116 B]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-bin amd64 2.4.58-1ubuntu8.4 [1,329 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-data all 2.4.58-1ubuntu8.4 [163 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-utils amd64 2.4.58-1ubuntu8.4 [97.1 kB]
```

Enable Apache: sudo systemctl enable apache2

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
```

Verify Apache is running: sudo systemctl status apache2

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo systemctl status apache2
[sudo] password for selcia:
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset:>
     Active: active (running) since Thu 2024-10-03 13:43:03 IST; 1h 31min ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 4373 (apache2)
      Tasks: 55 (limit: 4558)
     Memory: 5.8M (peak: 6.0M)
        CPU: 514ms
     CGroup: /system.slice/apache2.service
             ├─4373 /usr/sbin/apache2 -k start
             ├─4375 /usr/sbin/apache2 -k start
             └─4377 /usr/sbin/apache2 -k start

Oct 03 13:43:03 selcia-VMware-Virtual-Platform systemd[1]: Starting apache2.se>
Oct 03 13:43:03 selcia-VMware-Virtual-Platform apachectl[4372]: AH00558: apach>
Oct 03 13:43:03 selcia-VMware-Virtual-Platform systemd[1]: Started apache2.ser>
```

You can modify the Apache default web page to make it unique for each instance:
sudo nano /var/www/html/index.html

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo nano /var/www/html/index.
html
```

After customizing html template -> Ctrl O + Enter + Ctrl X

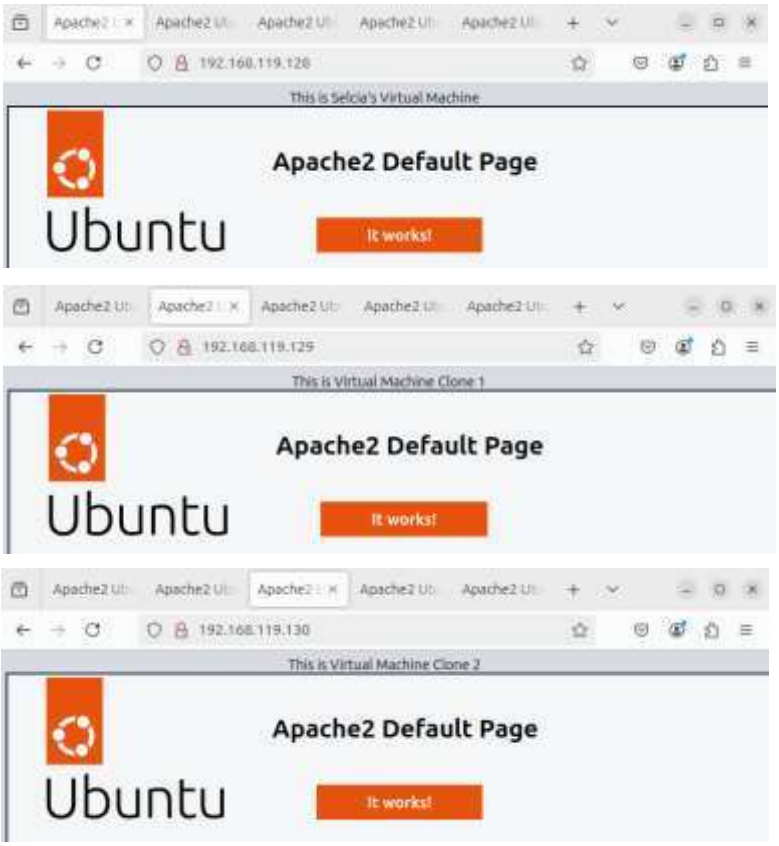
5. **Configure Networking:** ifconfig

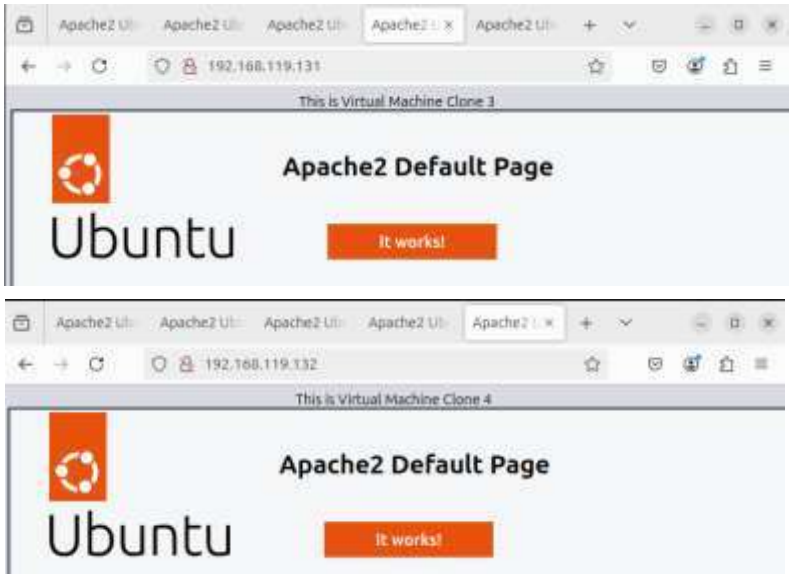
```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo apt install net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  net-tools
0 upgraded, 1 newly installed, 0 to remove and 31 not upgraded.
Need to get 204 kB of archives.
After this operation, 811 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu noble/main amd64 net-tools amd64 2.10-0.1ubuntu4 [204 kB]
Fetched 204 kB in 2s (105 kB/s)
Selecting previously unselected package net-tools.
(Reading database ... 149652 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-0.1ubuntu4_amd64.deb ...
Unpacking net-tools (2.10-0.1ubuntu4) ...
Setting up net-tools (2.10-0.1ubuntu4) ...
Processing triggers for man-db (2.12.0-4build2) ...
```

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.119.129  netmask 255.255.255.0  broadcast 192.168.119.255
        inet6 fe80::20c:29ff:fe16:b99e  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:16:b9:9e  txqueuelen 1000  (Ethernet)
        RX packets 1971  bytes 2578003 (2.5 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 620  bytes 52963 (52.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 197  bytes 20042 (20.0 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 197  bytes 20042 (20.0 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

OUTPUT:





Task 2: Implement Basic Load Balancing

1. **Install and Configure HAProxy on a Separate VM (Load Balancer):**
    - You can use a separate VM to act as the load balancer or install HAProxy on one of your VMs.

Install HAProxy using the following command: sudo apt install haproxy -y

2. **Configure HAProxy:**

Open the HAProxy configuration file: sudo nano /etc/haproxy/haproxy.cfg

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo nano /etc/haproxy/haproxy
.cfg
```

Add the following configuration to set up load balancing among your VMs:

```
frontend http_front
   bind *:80
   default_backend http_back

backend http_back
   balance roundrobin
   server vm1 <VM1_IP>:80 check
   server vm2 <VM2_IP>:80 check
   server vm3 <VM3_IP>:80 check
```

```
frontend http_front
        bind *:80
        default_backend http_back

backend http_back
        balance roundrobin
        server webserver1 192.168.119.128:80 check
        server webserver2 192.168.119.129:80 check
```

3. **Start HAProxy Service:**

After configuring HAProxy, restart the service to apply changes: sudo systemctl restart haproxy

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo systemctl restart haproxy
```

sudo systemctl enable haproxy

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo systemctl enable haproxy
Synchronizing state of haproxy.service with SysV service script with /usr/lib/s
ystemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable haproxy
```

sudo systemctl status haproxy

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ systemctl status haproxy
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; preset:>
     Active: active (running) since Thu 2024-10-03 15:28:13 IST; 13s ago
       Docs: man:haproxy(1)
             file:/usr/share/doc/haproxy/configuration.txt.gz
   Main PID: 9156 (haproxy)
     Status: "Ready."
      Tasks: 3 (limit: 4558)
     Memory: 38.9M (peak: 39.4M)
        CPU: 156ms
     CGroup: /system.slice/haproxy.service
             ├─9156 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/>
             └─9158 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/>

Oct 03 15:28:13 selcia-VMware-Virtual-Platform systemd[1]: Starting haproxy.se>
Oct 03 15:28:13 selcia-VMware-Virtual-Platform haproxy[9156]: [NOTICE]   (9156>
Oct 03 15:28:13 selcia-VMware-Virtual-Platform haproxy[9156]: [NOTICE]   (9156>
Oct 03 15:28:13 selcia-VMware-Virtual-Platform systemd[1]: Started haproxy.ser>
```

4. **Test the Load Balancer:** http://<load_balancer_IP>

OUTPUT:

http://192.168.119.128







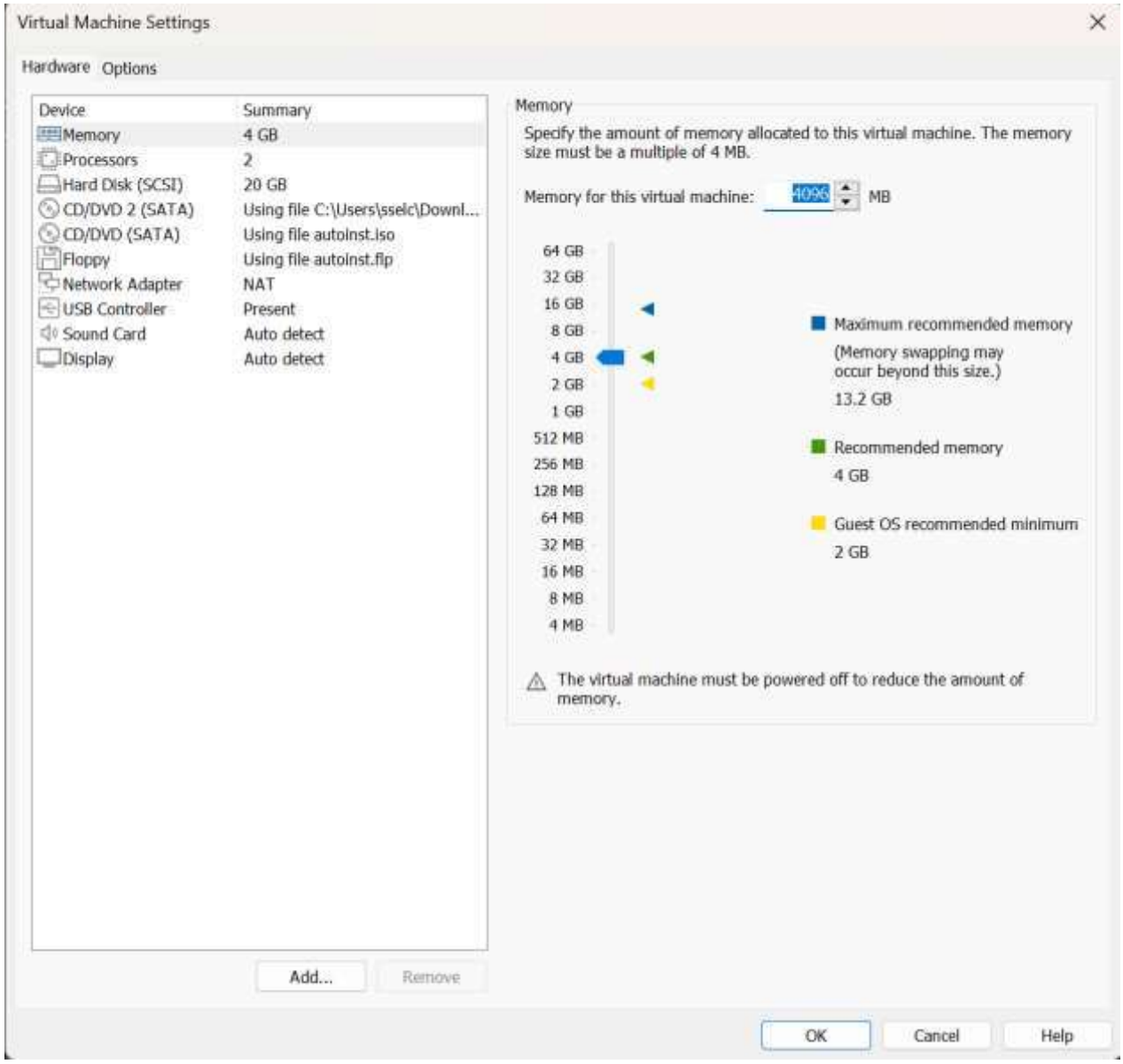


Part 2: Server Consolidation

Task 1: Implement Server Consolidation Strategy

1. **Open VMware Workstation and Identify VMs:**

- Launch VMware Workstation and review all VMs by opening each VM's configuration to check its resource allocations.



- List VMs with low usage or redundant workloads that could be consolidated.

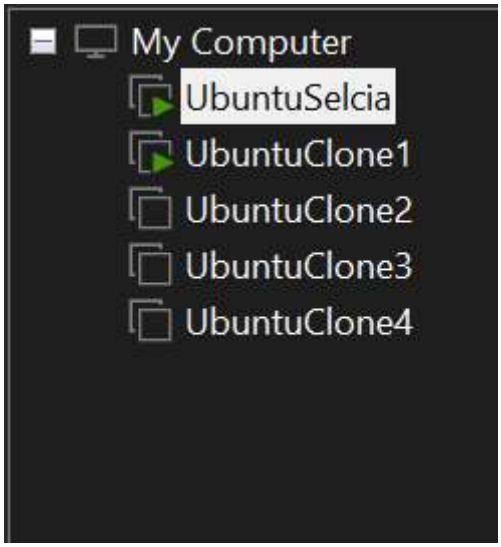
2. **Select a Consolidation Host:**
   - Choose a host VM with sufficient CPU, memory, and storage to accommodate the combined workloads.
3. **Adjust Resources for Each VM:**
   - Open each VM setting by selecting the VM in VMware Workstation, clicking *Edit virtual machine settings*, and adjusting *Memory* and *Processor* values to optimize the combined workload.
   - Avoid over-provisioning to ensure balanced performance.

4. **Migrate Services to a Consolidated VM:**
    - For each VM to be consolidated, copy files and services to the target consolidated VM.
    - Start the services on the consolidated VM and confirm they are running as expected.
5. **Power Off and Decommission Unused VMs:**
    - After successfully migrating services, power off the original VMs to ensure the services are fully operational on the consolidated VM.
    - Delete unused VMs by right-clicking the VM in VMware Workstation and selecting *Remove*.
6. **Validate Resource Utilization on the Consolidated VM:**
    - Check the resource usage (CPU, Memory) by accessing the *Performance Monitor* (for Windows) or using tools like top or htop (for Linux).
    - Verify that the consolidated VM can handle the load without performance degradation.

Part 3: Fault Tolerance

Task 1: Implement Fault-Tolerant Design

1. **Create Redundant VM Copies for High Availability:**
    - For critical services, create a clone of the primary VM by selecting the VM in VMware Workstation, going to *VM* > *Manage* > *Clone*, and choosing *Full Clone* to make a complete backup.



2. **Set Up Data Synchronization:**

For Linux, use rsync to sync files between primary and backup VMs by running a command like: rsync -av /source_directory/ user@backup_vm_ip:/destination_directory

3. **Enable Snapshots for Quick Recovery:**
    - In VMware Workstation, select a critical VM, go to *VM* > *Snapshot* > *Take Snapshot*, and give it a name (e.g., "Pre-Consolidation Snapshot").
    - Take snapshots before major updates or configurations to quickly revert if needed.

RESULT:

Virtual instances with load balancing, server consolidation, and fault tolerance were successfully deployed, optimizing performance and ensuring service continuity.

UIT2702 - Cloud and Distributed Computing

Exercise 6: Task Scheduling and Load Balancing in a Cloud Environment

AIM:

To automate task scheduling and implement load balancing in a cloud environment for efficient resource management and high availability.

PROCEDURE:

Part 1: Task Scheduling

Task 1: Implement Task Scheduling with Cloud Services

1. **Open VMware Workstation Pro 17**:
   - Power on the VMs (e.g., UbuntuSelcia).
2. **Configure Task Scheduling on Ubuntu VMs**:
   - We will use cron to schedule tasks, which is the Linux utility for task scheduling.
3. **Create a Task in Ubuntu VM**:

Create a script file using: nano script.sh



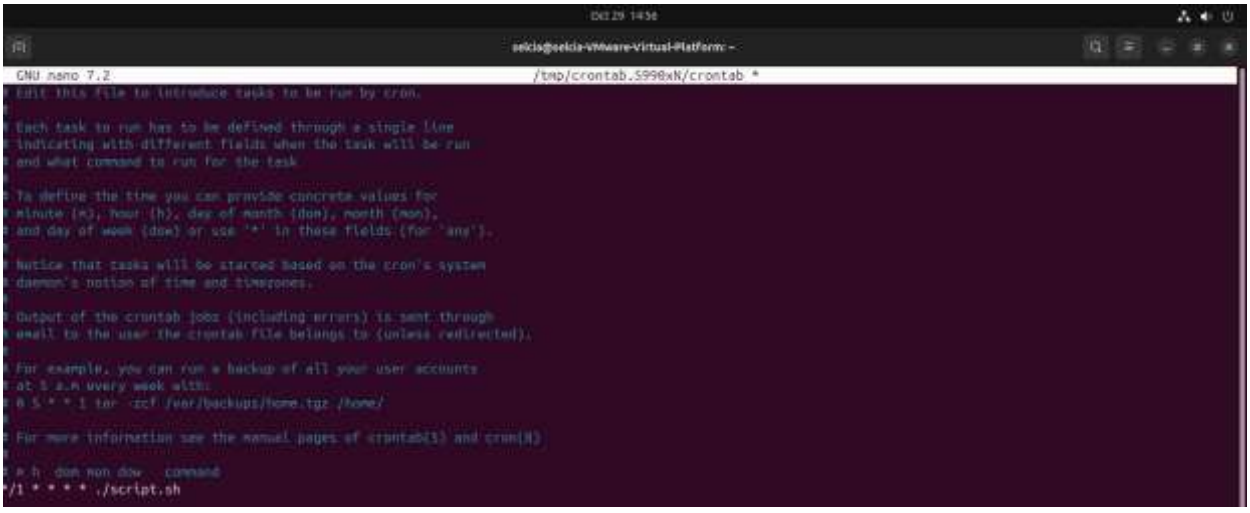
Add: echo “Hello Selcia”



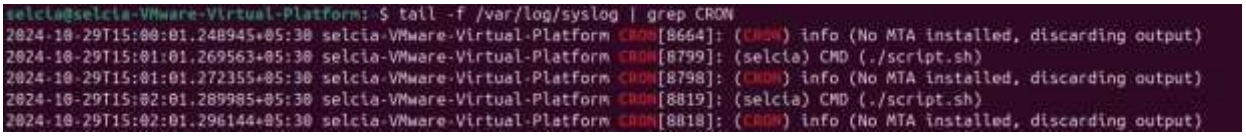
Open the cron file using the command: crontab -e



Add a scheduled task (for example, to run a script every 1 minute): */1 * * * * ./script.sh >> ./output.log



4. **Verify the Task**:
    - Check if the task is running by checking the output of the script or inspecting logs, if any: tail -f /var/log/syslog | grep CRON



OUTPUT:



Part 2: Load Balancing

Task 1: Implement Load Balancing

1. **Install and Configure HAProxy on a Separate VM (Load Balancer):**
    - You can use a separate VM to act as the load balancer or install HAProxy on one of your VMs.

Install HAProxy using the following command: sudo apt install haproxy -y

2. **Configure HAProxy:**

Open the HAProxy configuration file: sudo nano /etc/haproxy/haproxy.cfg



Add the following configuration to set up load balancing among your VMs:

```
frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server vm1 <VM1_IP>:80 check
    server vm2 <VM2_IP>:80 check
    server vm3 <VM3_IP>:80 check
```

```
frontend http_front
        bind *:80
        default_backend http_back

backend http_back
        balance roundrobin
        server webserver1 192.168.119.128:80 check
        server webserver2 192.168.119.129:80 check
```

3. **Start HAProxy Service:**

After configuring HAProxy, restart the service to apply changes: sudo systemctl restart haproxy

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo systemctl restart haproxy
```

sudo systemctl enable haproxy

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ sudo systemctl enable haproxy
Synchronizing state of haproxy.service with SysV service script with /usr/lib/s
ystemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable haproxy
```

sudo systemctl status haproxy

```
selcia@selcia-VMware-Virtual-Platform:~/Desktop$ systemctl status haproxy
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; preset:>
     Active: active (running) since Thu 2024-10-03 15:28:13 IST; 13s ago
       Docs: man:haproxy(1)
             file:/usr/share/doc/haproxy/configuration.txt.gz
   Main PID: 9156 (haproxy)
     Status: "Ready."
      Tasks: 3 (limit: 4558)
     Memory: 38.9M (peak: 39.4M)
        CPU: 156ms
     CGroup: /system.slice/haproxy.service
             ├─9156 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/>
             └─9158 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/>

Oct 03 15:28:13 selcia-VMware-Virtual-Platform systemd[1]: Starting haproxy.se>
Oct 03 15:28:13 selcia-VMware-Virtual-Platform haproxy[9156]: [NOTICE]   (9156>
Oct 03 15:28:13 selcia-VMware-Virtual-Platform haproxy[9156]: [NOTICE]   (9156>
Oct 03 15:28:13 selcia-VMware-Virtual-Platform systemd[1]: Started haproxy.ser>
```

4. **Test the Load Balancer:** http://<load_balancer_IP>

OUTPUT:

http://192.168.119.128









Part 3: Advanced Scheduling and Load Balancing (Optional)

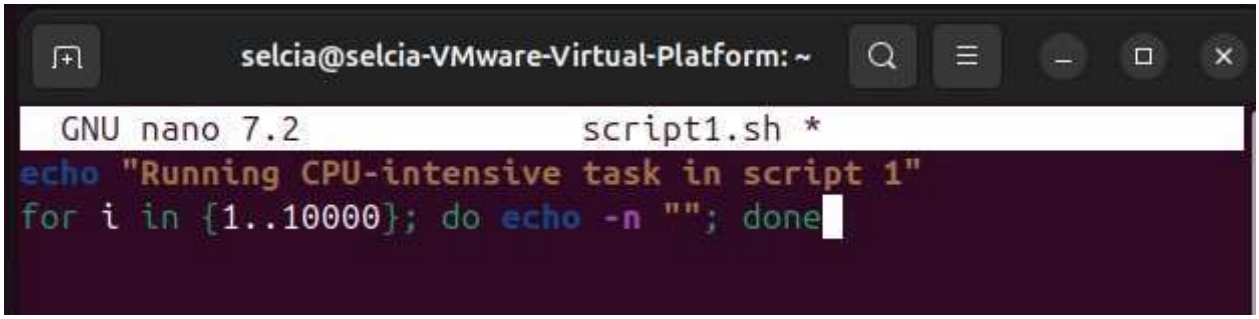
Task 1: Implement Advanced Scheduling with Prioritization

1. **Prioritization in Task Scheduling (Cron)**:

Make several script files with different tasks.

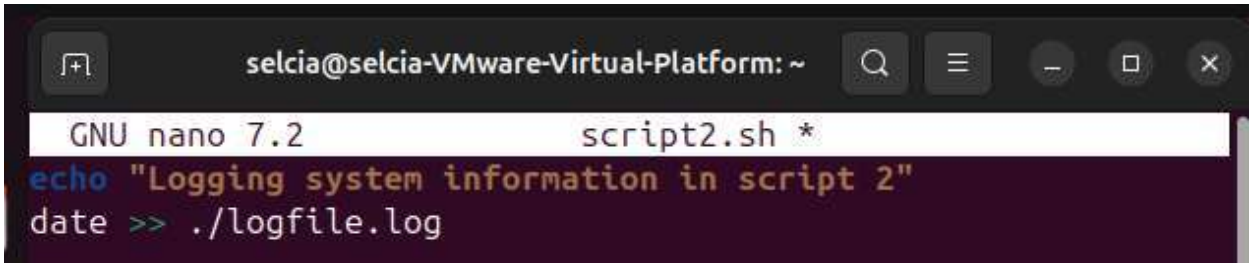
script1.sh: Could simulate a CPU-intensive task (e.g., a loop or heavy computation).



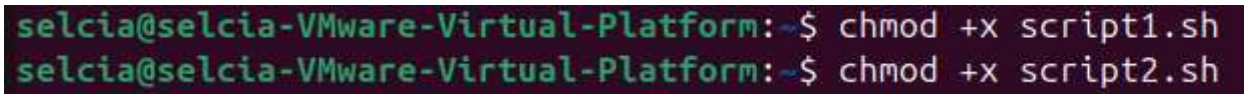


script2.sh: Could simulate a lighter task (e.g., logging system information).

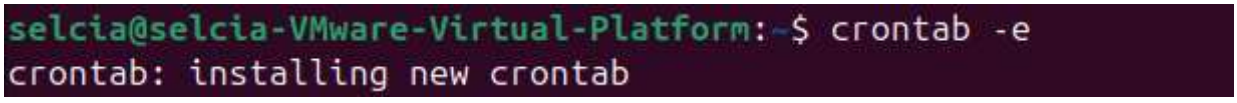




Make the Scripts Executable: Give execute permissions to the scripts.



Schedule Cron Jobs with Nice Values: Edit your crontab



Add the following lines to set different priority levels:

```
*/1 * * * * nice -n 10 ./script1.sh >> ./output-pr.log
*/1 * * * * nice -n 5 ./script2.sh >> ./output-pr.log
```

With nice, a lower priority (higher nice value) means the task will receive CPU time only when the system is idle or has available CPU cycles. By contrast, a higher priority (lower nice value) allows the task to execute more readily.

```
selcia@selcia-VMware-Virtual-Platform:~$ cat output-pr.log
Logging system information in script 2
Running CPU-intensive task in script 1
```

In this case, nice -n 10 lowers the priority of the task. You can adjust the priority by changing the value between -20 (highest priority) and 19 (lowest priority).

RESULT:

Virtual instances with load balancing, server consolidation, and fault tolerance were successfully deployed, optimizing performance and ensuring service continuity.

UIT2702 - Cloud and Distributed Computing
Exercise 7: Hypervisor Clustering in a Cloud Environment

AIM:

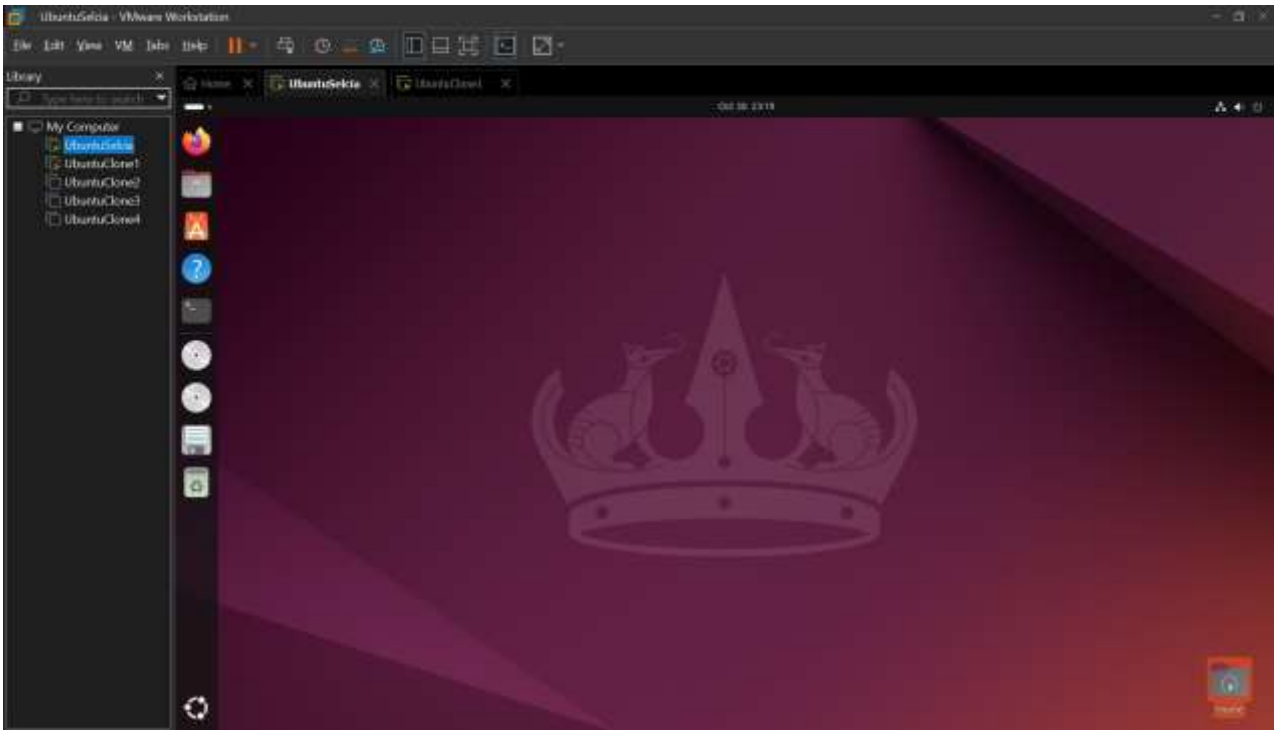
To implement hypervisor clustering in a cloud environment.

PROCEDURE:

Part 1: Setting Up a Hypervisor Cluster

Task 1: Set Up Hypervisor Hosts

1. **Create Virtual Machines for Each Hypervisor Host:**
    - In VMware Workstation, click **File > New Virtual Machine**.



    - Follow the wizard to create two or more virtual machines that will act as hypervisor hosts. Install **VMware ESXi**, **Proxmox VE**, or another hypervisor-compatible OS on each.
    - Assign appropriate resources (e.g., 2 CPUs, 8 GB RAM) to each hypervisor host to simulate a realistic cluster.
2. **Configure Network Settings for VM Communication:**
    - Set each hypervisor VM's **network adapter** to **Bridged mode** so that each VM can communicate on the same network as the host machine.
3. **Install and Configure the Hypervisors:**

- Install Proxmox on each VM.
- After installation, access each hypervisor VM through a web interface or console and complete any additional setup (e.g., IP configuration, root password).

Task 2: Configure Hypervisor Clustering

1. **Enable Cluster Features on Each Hypervisor:**
   - On each hypervisor (ESXi, Proxmox, etc.), enable clustering and high availability features, if available. For Proxmox:
     - In the Proxmox web interface, navigate to **Datacenter > Cluster > Create Cluster** on the first hypervisor and note the cluster join information.
     - On the second hypervisor, join the cluster using the details from the first hypervisor.
2. **Verify Cluster Setup:**
   - Confirm that each hypervisor node can see the other nodes in the cluster.
   - Test network connectivity between the nodes using ping commands to ensure they can communicate.

Part 2: Implement High Availability and Load Balancing

Task 1: Configure High Availability for VMs

1. **Enable High Availability in Cluster Settings:**
   - In the hypervisor's management interface, navigate to the **High Availability** settings for your cluster.
   - Specify which VMs should be automatically restarted on another hypervisor in the event of a host failure.
2. **Test High Availability:**
   - Power down one of the hypervisor hosts and verify that the VM configured with high availability is automatically restarted on another hypervisor.

Task 2: Implement Load Balancing

1. **Configure Load Balancing Policies:**
   - In the hypervisor's settings, configure **resource distribution policies** to balance the CPU and memory usage across the cluster.
   - Some platforms allow setting thresholds to move VMs to other hosts when resource usage exceeds defined limits.
2. **Monitor and Test Load Balancing:**
   - Observe the distribution of VM workloads in your hypervisor's dashboard.
   - Optionally, run CPU and memory-intensive tasks on VMs to simulate load and check if the system balances the load.

Part 3: Manage and Migrate Virtual Machines

Task 1: Migrate VMs Across Hypervisors

1. **Prepare for Migration:**
    - Ensure the VMs to be migrated are configured for **shared storage** and **networking** so they can operate across multiple hosts.
    - VMware Workstation does not support live migration by default, but you can power off a VM and move it manually between hosts as a simulation.
2. **Perform Manual Migration:**
    - Power off the VM, then export it as an **OVF template** or copy its virtual disk files to another hypervisor node.
    - Import the VM on the new hypervisor host and start it up to complete the migration process.
3. **Verify Migration:**
    - After starting the VM on the new host, verify that it's functioning as expected and connected to the same network as before.

RESULT:

Hypervisor clustering has been successfully implemented in VMWare Workstation Pro 17.